

Study Group Platform - MERN Stack

Responsibility Matrix & Component Division

BSc (Hons) IT Year 3 - IT3040 ITPM

Project Overview

An online learning platform for university students featuring mock exams, lecture resources, Kuppi sessions, progress tracking, quiz management, AI chatbot support, and payment gateway integration for premium content.

Technology Stack: MERN (MongoDB, Express.js, React.js, Node.js)

Team Size: 4 Members

Project Duration: 11 Weeks

Member 1: Course & Content Management System

Student Name	[Member 1 Name]
Registration Number	[IT Number]
Component(s)	• Course Catalog & Enrollment System • Lecture Notes & Resource Management • Content Upload & Organization

Component Description:

Inputs:

- Student selects course category and subject area
- Student clicks on a course to view details and enrollment options
- Admin/Tutor uploads lecture notes, PDFs, videos, and learning materials
- Admin organizes content by course, topic, and week
- Student searches for specific lecture topics or materials

System Outputs:

- Display available courses with thumbnails, descriptions, and enrollment status
- Course enrolled successfully with confirmation message
- Lecture materials organized in folders by week/topic with download links
- Search results showing relevant notes and videos
- Recently accessed materials displayed on student dashboard

Intended User Outcomes:

- Students can easily discover and enroll in relevant courses
- Students have organized access to all lecture materials in one place
- Reduced time searching for study materials
- Better preparation for exams with structured content delivery

- Tutors can efficiently manage and share educational content

Technical Implementation:

- Frontend: React components for course listing, course detail pages, enrollment UI, file upload interface, resource browser
- Backend: Express.js API endpoints for course CRUD operations, file upload handling (Multer), enrollment management
- Database: MongoDB schemas for courses, enrollments, lecture materials, file metadata
- Integration: AWS S3 or Cloudinary for file storage, search functionality with MongoDB text indexes

Member 2: Quiz & Mock Exam System with Progress Tracking

Student Name	[Member 2 Name]
Registration Number	[IT Number]
Component(s)	• Mock Exam & Quiz Creation System • Quiz Taking Interface with Timer • Real-time Student Progress Tracking Dashboard

Component Description:

Inputs:

- Admin/Tutor creates quiz with title, subject, difficulty level, time limit
- Admin adds questions (MCQ, True/False, Short Answer) with correct answers and marks
- Student selects a quiz and clicks 'Attempt Quiz' button
- Student answers questions and submits quiz before time expires
- Admin views student progress analytics and performance metrics

System Outputs:

- Quiz saved in database with all questions and grading configuration
- Quiz interface displays questions one-by-one or all at once with countdown timer
- Automatic grading with instant score and detailed answer review
- Student results saved to database with timestamp and performance metrics
- Progress dashboard showing student performance graphs, quiz history, strengths/weaknesses
- Admin analytics showing class average, individual student performance, question-level statistics

Intended User Outcomes:

- Students can practice with realistic exam simulations
- Instant feedback helps students identify knowledge gaps

- Students can track their improvement over time
- Tutors can monitor student progress and intervene when needed
- Data-driven insights help both students and tutors optimize learning strategies

Technical Implementation:

- Frontend: React components for quiz builder, quiz taking UI with timer, results display, progress charts (Chart.js/Recharts)
- Backend: Express.js APIs for quiz CRUD, automatic grading logic, analytics aggregation
- Database: MongoDB schemas for quizzes, questions, student attempts, results, progress metrics
- Real-time: Socket.io for live quiz sessions (optional), timer synchronization

Member 3: Kuppi Session Booking & User Management

Student Name	[Member 3 Name]
Registration Number	[IT Number]
Component(s)	• Kuppi Session Scheduling & Booking • User Authentication & Authorization • User Profile Management

Component Description:

Inputs:

- Tutor creates Kuppi session with subject, date, time, duration, max participants
- Student browses available Kuppi sessions and selects preferred slot
- Student clicks 'Book Session' to register
- User registers/logs in with email, password
- User selects role (Student/Tutor) during registration
- User updates profile information, profile picture, bio, subjects of interest

System Outputs:

- Kuppi session created and visible in session calendar
- Booking confirmed with confirmation message and email notification
- Session details added to student's 'My Sessions' dashboard
- Tutor receives notification of new booking
- User authenticated with JWT token stored in browser
- Role-based access control enforced (Students cannot create quizzes, Tutors can)
- Profile information updated and displayed across platform

Intended User Outcomes:

- Students can easily find and book peer study sessions
- Reduced coordination time for group study
- Students receive reminders and avoid missing important sessions
- Secure access to platform with proper authentication

- Different user roles have appropriate access levels
- Personalized experience with user profiles and preferences

Technical Implementation:

- Frontend: React components for session calendar, booking form, login/register UI, profile pages
- Backend: Express.js APIs for session CRUD, booking management, JWT authentication, password hashing (bcrypt)
- Database: MongoDB schemas for sessions, bookings, users, roles
- Integration: Nodemailer for email notifications, middleware for role-based authorization

Member 4: AI Chatbot & Payment Gateway Integration

Student Name	[Member 4 Name]
Registration Number	[IT Number]
Component(s)	• AI-Powered Student Support Chatbot • Payment Gateway for Premium Content • Quiz Marketplace & Transaction Management

Component Description:

Inputs:

- Student types question in chatbot interface
- Chatbot receives context about student's current course, recent activities
- Student selects premium quiz/course to purchase
- Student enters payment details (card number, CVV, billing address)
- Student confirms payment and clicks 'Complete Purchase'

System Outputs:

- AI chatbot provides contextual answers about courses, study tips, platform help
- Chatbot suggests relevant resources, quizzes, or Kuppi sessions
- Chat history saved for continuity in future conversations
- Payment processed securely through Stripe/PayPal gateway
- Transaction confirmation with receipt sent to student email
- Premium content immediately unlocked in student's account
- Transaction record saved in database for admin revenue tracking

Intended User Outcomes:

- Students get instant help 24/7 without waiting for tutor response
- Reduced workload on tutors for repetitive questions
- Personalized study recommendations based on student performance
- Seamless purchase experience for premium study materials
- Monetization opportunity for quality quiz creators
- Secure and trustworthy payment processing

Technical Implementation:

- Frontend: React chatbot widget, payment checkout UI, transaction history page
- Backend: Express.js APIs for chatbot endpoints, Stripe/PayPal integration, webhook handlers
- AI Integration: OpenAI GPT API or Dialogflow for natural language processing
- Database: MongoDB schemas for chat history, transactions, purchased content
- Security: PCI compliance, encrypted payment data, webhook signature verification

Quick Reference Summary

Member	Primary Focus	Key Technologies
Member 1	Content & Course Management	React, Express, MongoDB, Multer, AWS S3/Cloudinary
Member 2	Quiz System & Analytics	React, Chart.js, Express, MongoDB, Socket.io (optional)
Member 3	Booking & User Management	React, JWT, Bcrypt, Express, MongoDB, Nodemailer
Member 4	AI Chatbot & Payments	React, OpenAI API/Dialogflow, Stripe/PayPal, Express, MongoDB

Shared Responsibilities (All Members)

- Version control using Git with meaningful commits
- Project management using Trello/Jira/GitHub Projects
- Automated testing (Cypress for web application)
- Input validation for all forms and APIs
- Responsive UI design (mobile and desktop)
- Error handling and user feedback
- Documentation of APIs and components

Key Integration Points

Member 1 ↔ Member 2: Quizzes linked to specific courses and lecture materials

Member 2 ↔ Member 3: Quiz results displayed in user profiles, authentication required for quiz attempts

Member 3 ↔ Member 4: Payment integration for premium Kuppi sessions, user authentication for chatbot personalization

Member 4 ↔ Member 1: Payment gateway for purchasing premium courses and materials

All Members: Shared MongoDB database, common React component library, unified API structure

Suggested Development Timeline (11 Weeks)

Weeks 1-2: Requirements analysis, database design, API planning, initial setup

Weeks 3-6: Core feature development for each member's component

Weeks 7-8: Integration of components, testing, bug fixes

Weeks 9-10: UI/UX refinement, performance optimization, comprehensive testing

Week 11: Final testing, documentation, deployment, presentation preparation

This division ensures each member has a substantial, independently demonstrable component with clear inputs, outputs, and user outcomes as required by the project guidelines.